

MobileNetV3 GroupNorm investigation

Following up on requests from the review: the cause of MobileNetV3's failure to learn has been identified and fixed.

Root cause. Opacus requires replacing BatchNorm with GroupNorm for privacy compliance before it can wrap a model in DP-SGD. `ModuleValidator.fix()` does this automatically, but it creates the new GroupNorm layers with default weights rather than carrying over anything from the pretrained BatchNorm layers. When these layers fall inside the frozen part of the backbone, those default, untrained values are locked in for the whole run and never get a chance to adjust, so the pretrained backbone's early layers are effectively feeding the rest of the network through an untrained normalization layers.

Steps taken to isolate this:

- Checked the group counts chosen by `ModuleValidator.fix()`, confirmed that some groups have size 1.
- Ran MobileNetV3 on CIFAR-10 without Opacus at all (no DP-SGD), keeping the Flower federated pipeline, and it converged well there.
- Applied `ModuleValidator.fix()` and confirmed that it introduces the model deterioration.
- Checked whether the group counts chosen by `ModuleValidator.fix()` were the problem (the bad divisor concern) by forcing a larger group size everywhere. This did not meaningfully improve accuracy, ruling out group size as the main cause.
- Tested unfreezing the GroupNorm weights, while keeping the rest of the backbone frozen as before. This substantially recovered accuracy (up to 70%, vs 35-45% before), showing the frozen default weights is the actual issue.
- Repeated the same test with Opacus enabled and DP noise and clipping disabled in the pipeline. Results were nearly identical to previous step, confirming that DP-SGD code works correct.
- Repeated the MobileNetV3 test that failed previous time.

GroupNorm substitution check

After running `ModuleValidator.fix()` on MobileNetV3, all BatchNorm2d layers were replaced with GroupNorm. This table lists, for each replaced layer: the channel count (unchanged, equals to BatchNorm2d `num_features` from before the fix), the `num_groups` value Opacus ModuleValidator chose, and the resulting group size (`channels / groups`).

```
In [1]: import opacus.validators, pandas, torch, torchvision.models
model = torchvision.models.mobilenet_v3_small()
model = opacus.validators.ModuleValidator.fix(model)
group_norm = []
for name, module in model.named_modules():
    if isinstance(module, torch.nn.GroupNorm):
        group_norm.append({
            "layer": name,
            "channels": module.num_channels,
            "num_groups": module.num_groups,
            "group size": module.num_channels // module.num_groups
        })
pandas.DataFrame(group_norm)
```

```
Out[1]:
```

	layer	channels	num_groups	group size
0	features.0.1	16	16	1
1	features.1.block.0.1	16	16	1
2	features.1.block.2.1	16	16	1
3	features.2.block.0.1	72	8	9
4	features.2.block.1.1	72	8	9
5	features.2.block.2.1	24	8	3
6	features.3.block.0.1	88	8	11
7	features.3.block.1.1	88	8	11
8	features.3.block.2.1	24	8	3
9	features.4.block.0.1	96	32	3
10	features.4.block.1.1	96	32	3
11	features.4.block.3.1	40	8	5
12	features.5.block.0.1	240	16	15
13	features.5.block.1.1	240	16	15
14	features.5.block.3.1	40	8	5
15	features.6.block.0.1	240	16	15
16	features.6.block.1.1	240	16	15
17	features.6.block.3.1	40	8	5
18	features.7.block.0.1	120	8	15

	layer	channels	num_groups	group size
19	features.7.block.1.1	120	8	15
20	features.7.block.3.1	48	16	3
21	features.8.block.0.1	144	16	9
22	features.8.block.1.1	144	16	9
23	features.8.block.3.1	48	16	3
24	features.9.block.0.1	288	32	9
25	features.9.block.1.1	288	32	9
26	features.9.block.3.1	96	32	3
27	features.10.block.0.1	576	32	18
28	features.10.block.1.1	576	32	18
29	features.10.block.3.1	96	32	3
30	features.11.block.0.1	576	32	18
31	features.11.block.1.1	576	32	18
32	features.11.block.3.1	96	32	3
33	features.12.1	576	32	18

```
In [2]: import pandas
import matplotlib.pyplot as plt

def flwr_accuracy(folder, key, title, exclude_ids = {}):
    df = pandas.read_csv(folder + "/flwr.csv")
    df = df[~df["id"].isin(exclude_ids)]
    plt.figure(figsize=(8,4))
    for id_val, g in df.groupby("id", sort=False):
        g = g.sort_values("round")
        plt.plot(g["round"], g[key], marker="o", label=str(id_val))

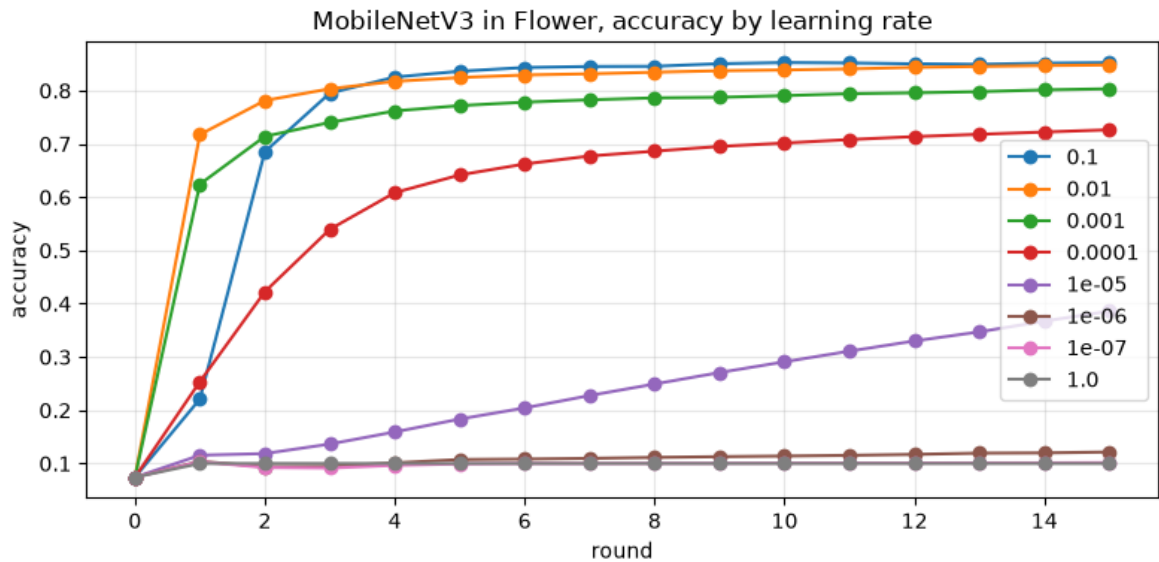
    plt.xlabel("round")
    plt.ylabel(key)
    plt.title(title)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()
```

MobileNetV3 in Flower federation

Test of default MobileNetV3 in Flower to ensure it gives the expected accuracy.

Unfrozen: classifier head plus the last layer of the pretrained backbone,
tune_layers: 1, batch-size: 32

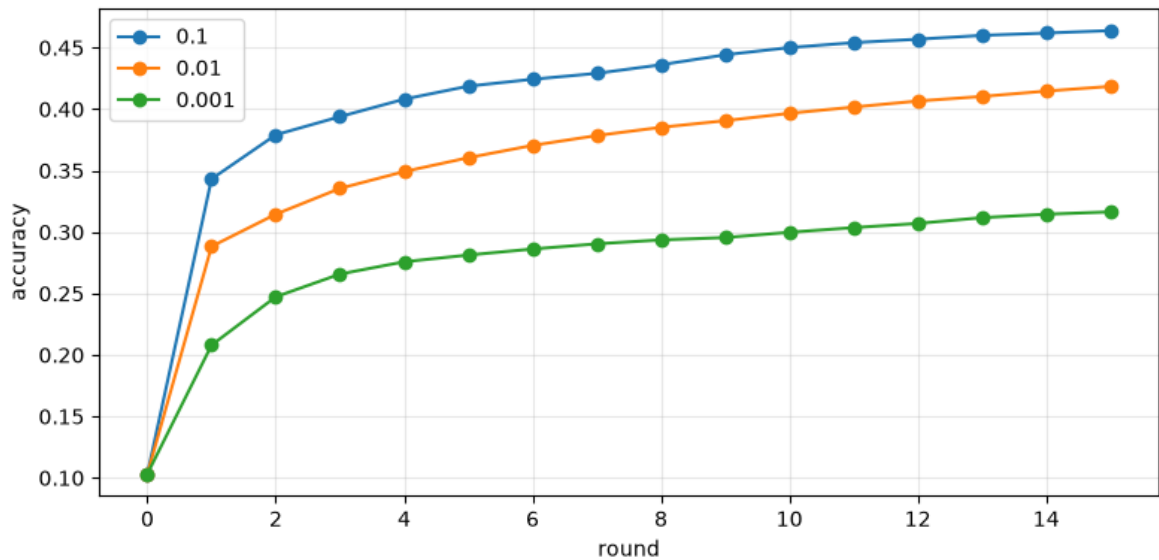
```
In [3]: flwr_accuracy("mnv3woo_lr", "accuracy", "MobileNetV3 in Flower, accuracy")
```



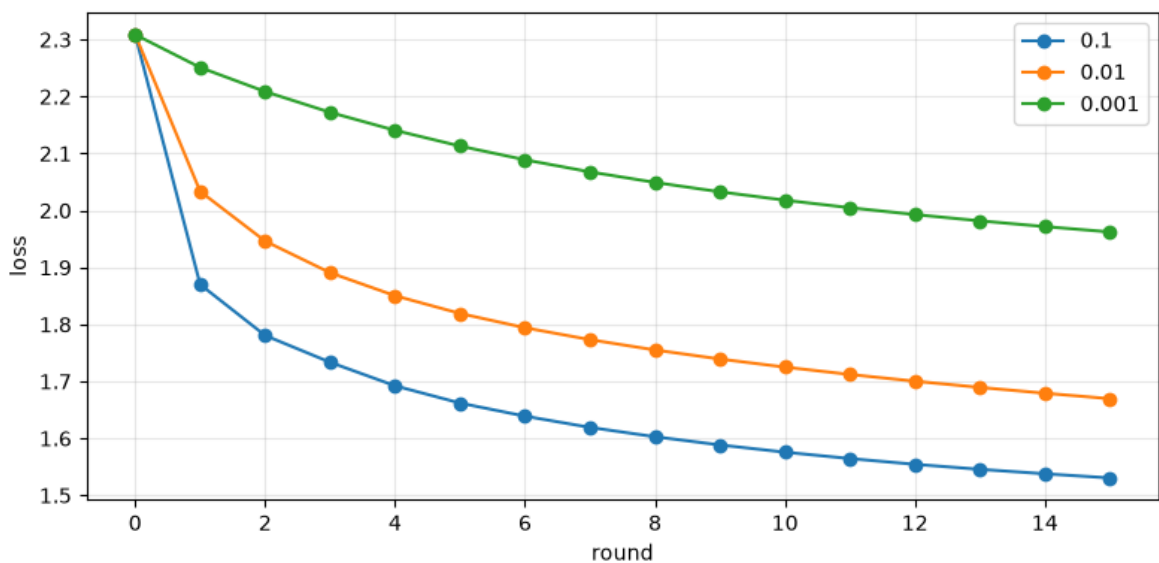
MobileNetV3 with applied ModuleValidator.fix(model)

The same default MobileNetV3 with applied ModuleValidator.fix(model),
tune_layers: 1, batch-size: 32

```
In [5]: flwr_accuracy("mnv3_groupnorm_lr/1", "accuracy", "")
```



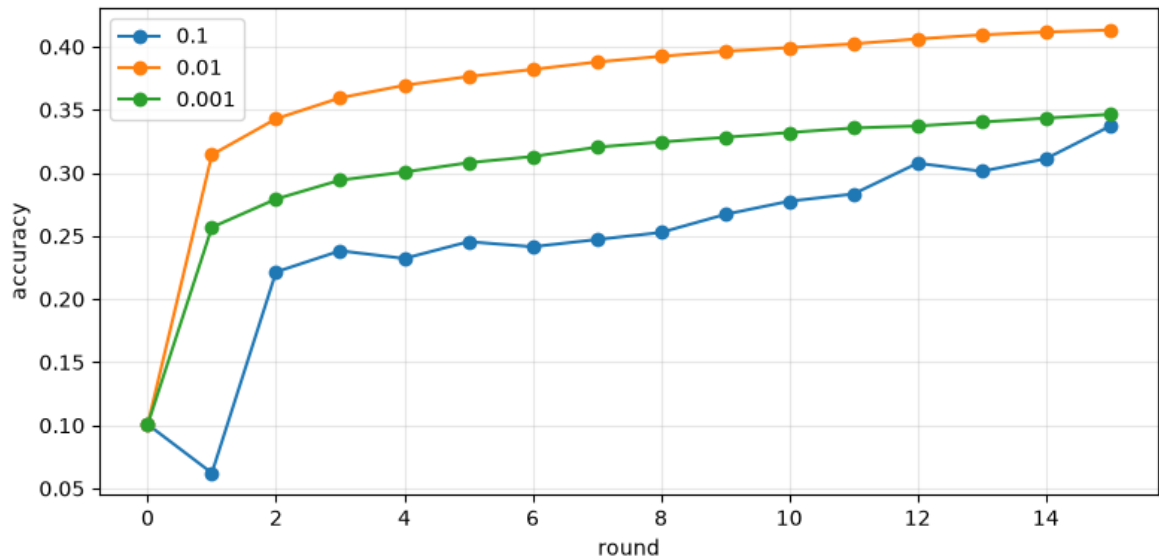
```
In [6]: flwr_accuracy("mnv3_groupnorm_lr/1", "loss", "")
```



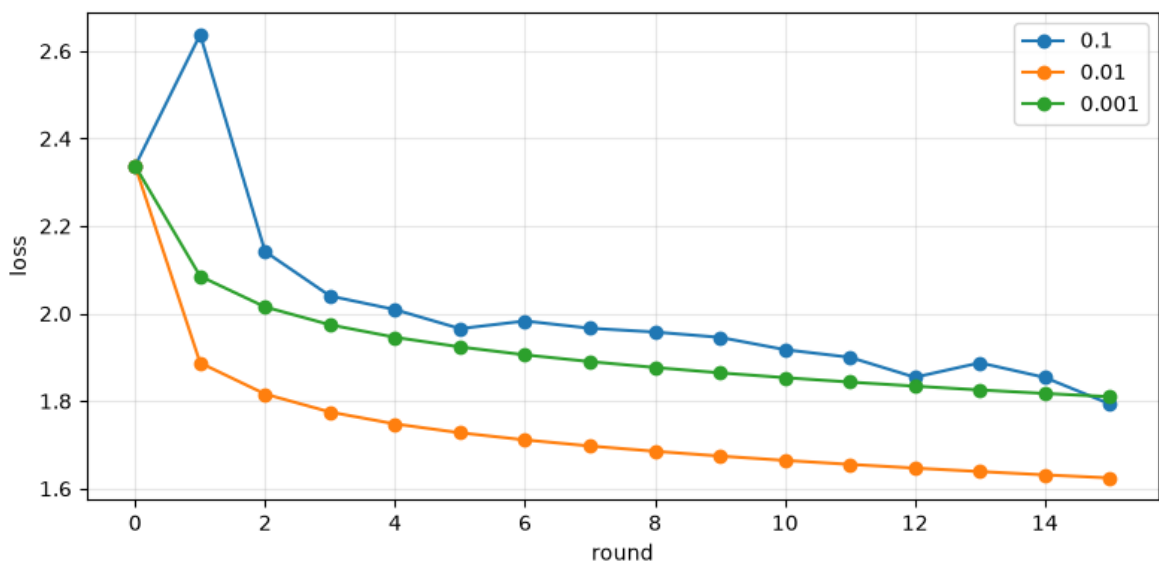
MobileNetV3 with applied ModuleValidator.fix(model, num_groups=4)

Applying ModuleValidator.fix() results in some groups having only one channel per group. This test eliminates this edge case.

```
In [7]: flwr_accuracy("mnv3_groupnorm_lr/2", "accuracy", "")
```



```
In [8]: flwr_accuracy("mnv3_groupnorm_lr/2", "loss", "")
```

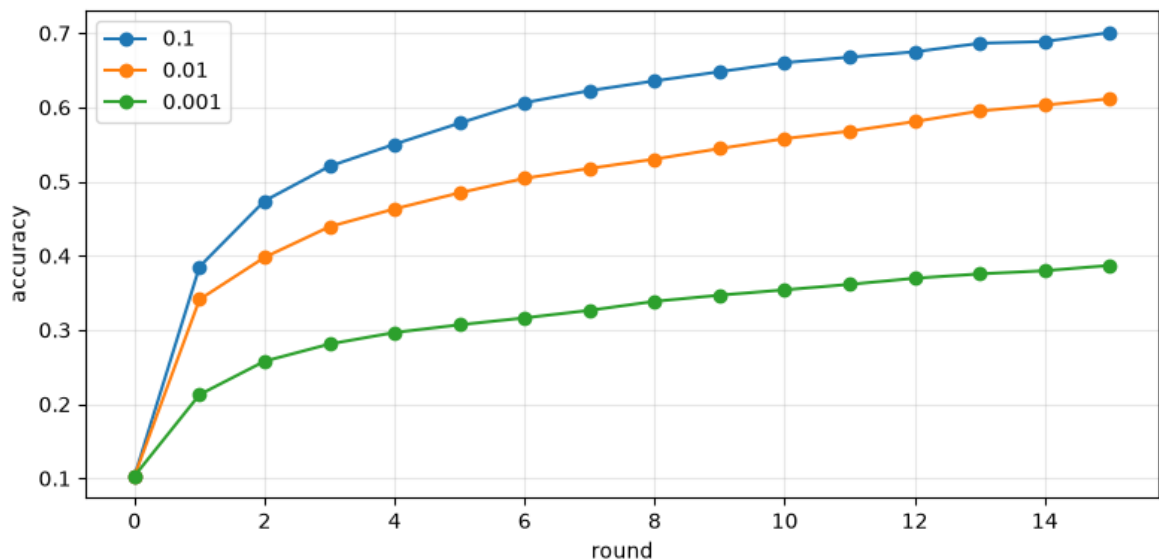


MobileNetV3 with applied ModuleValidator.fix(model) and all GroupNorm unfrozen

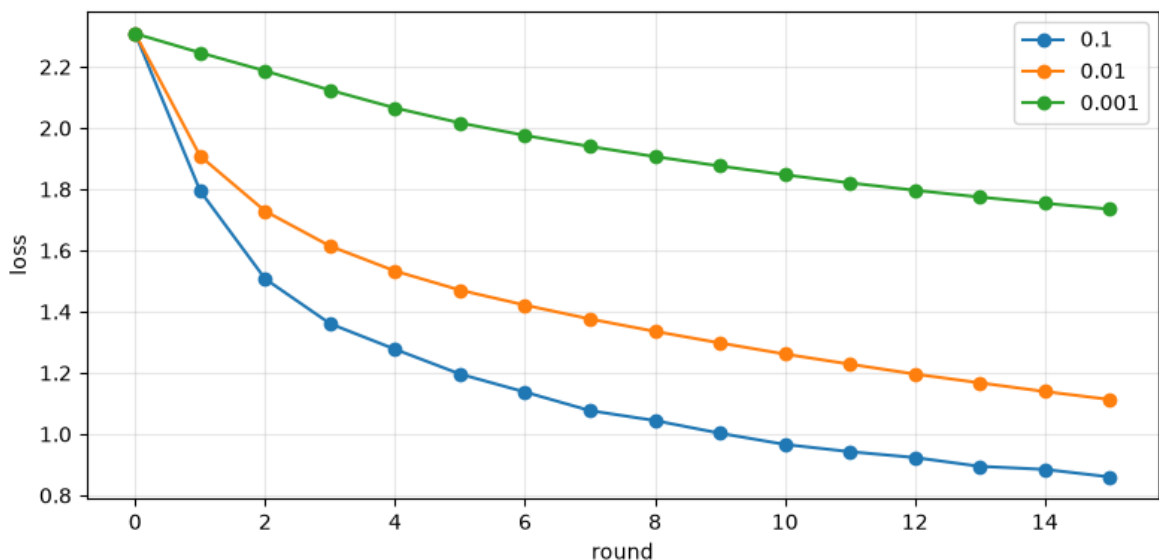
It is possible that BatchNorm2d layers lose their pretrained weights after being replaced with GroupNorm and need to be trained from scratch.

Unfrozen: classifier head plus the last layer of the pretrained backbone plus all GroupNorm layers.

```
In [9]: flwr_accuracy("mnv3_gn2_lr", "accuracy", "")
```



```
In [10]: flwr_accuracy("mnv3_gn2_lr", "loss", "")
```

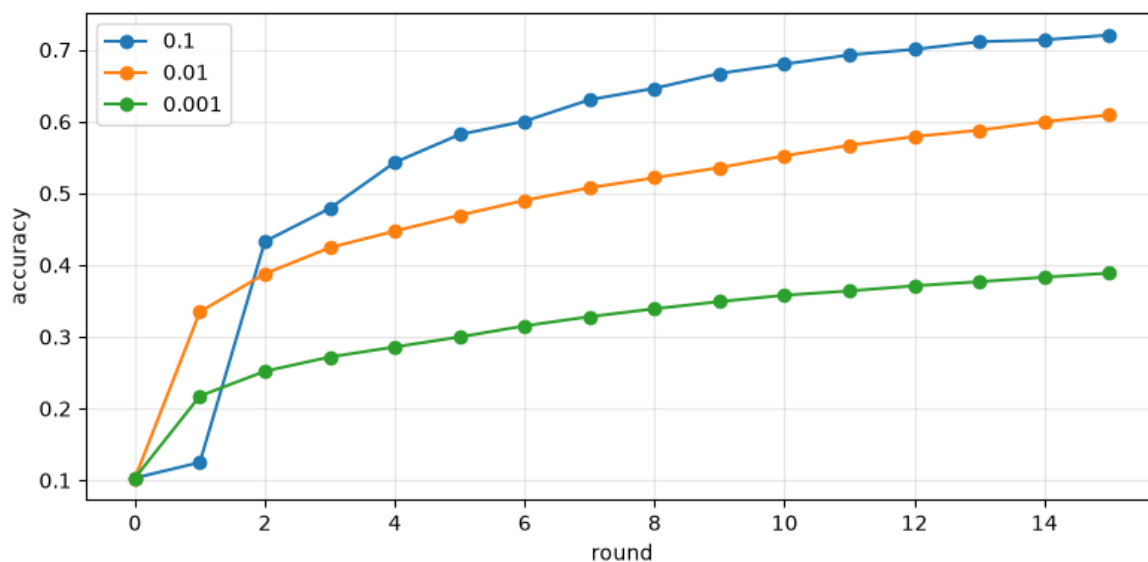


MobileNetV3 with all GroupNorm unfrozen in Opacus pipeline

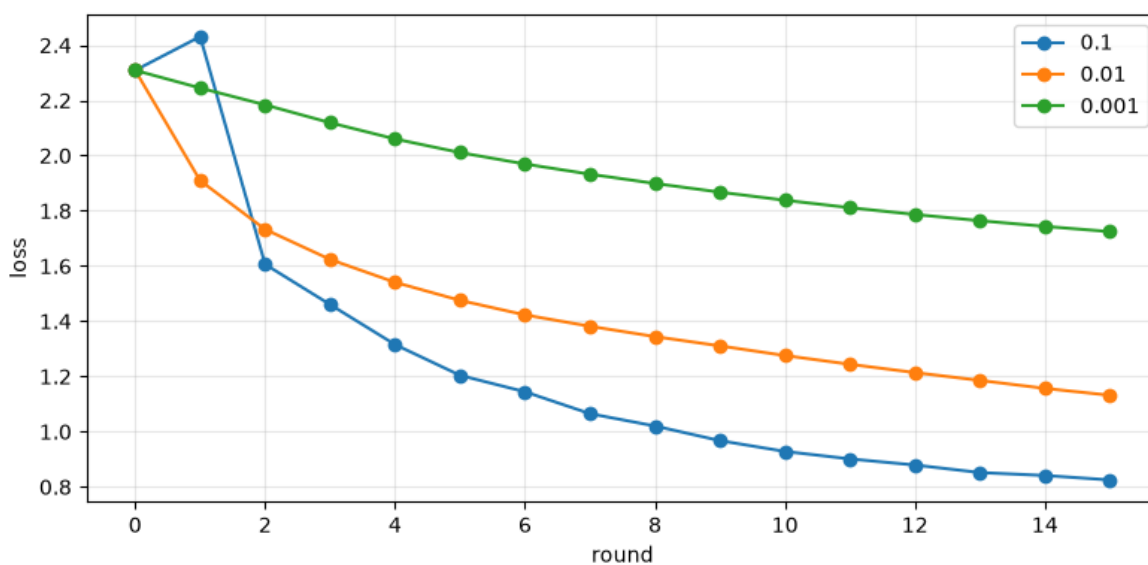
using settings to disable noise and clipping

noise-multiplier: 0, max-grad-norm: 1e90

```
In [11]: flwr_accuracy("mnv3_gn3_lr", "accuracy", "")
```



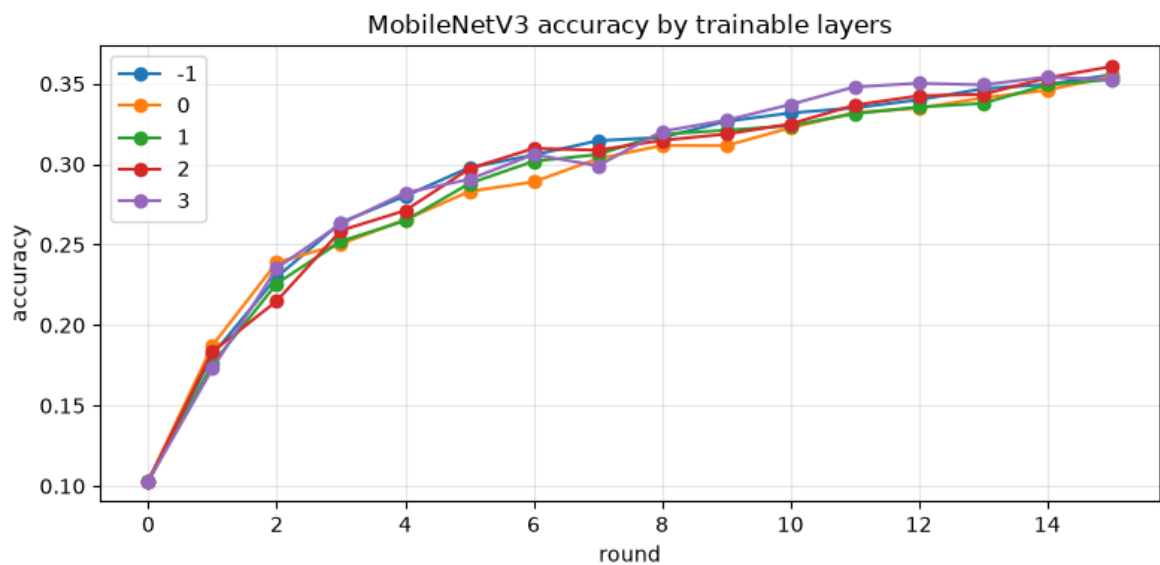
```
In [12]: flwr_accuracy("mnv3_gn3_lr", "loss", "")
```



MobileNetV3, fixed

noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.01, batch-size: 32

```
In [13]: flwr_accuracy("mn_layers/new", "accuracy", "MobileNetV3 accuracy by train
```



```
In [14]: flwr_accuracy("mn_layers/new", "loss", "")
```

